

Jan 21,2026

TESTING REPORT

Report Contents

- 1 Executive Summary
- 2 Backend API Test Results
- 3 Frontend UI Test Results
- 4 Analysis & Fix Recommendations

This report provides key insights from TestSprite's AI-powered testing. For questions or customized needs, contact us using [Calendly](#) or join our [Discord](#) community.

Table of Contents

Executive Summary

- 1 High-Level Overview
- 2 Key Findings

Backend API Test Results

- 3 Test Coverage Summary
- 4 Test Execution Summary
- 5 Test Execution Breakdown

Frontend UI Test Results

- 6 Test Coverage Summary
- 7 Test Execution Summary
- 8 Test Execution Breakdown

Executive Summary

1 High-Level Overview

OVERVIEW			
Total APIs Tested	1 APIs		
Total Websites Tested	1 Websites		
Pass/Fail Rate	Backend: 3/12 Frontend: 0/12		

2 Key Findings

Test Summary

The project demonstrates a clear disparity in quality between backend and frontend components, with backend performance appearing stable but lacking detailed insights. The absence of frontend testing results leaves significant gaps in evaluating user interface reliability, affecting overall user experience and stability.

What could be better

Frontend testing results are completely missing, highlighting a significant gap in assessing the reliability of user interactions. While there are no observed patterns of failure, the uncertainty regarding frontend performance presents risks to overall project stability.

Recommendations

Establish a comprehensive strategy for frontend testing to ensure user interface reliability. Regular backend performance evaluations should continue, focusing on addressing any deficiencies that may negatively impact user experience.

Backend API Test Results

3 Test Coverage Summary

API NAME	TEST CASES	TEST CATEGORY	PASS/FAIL RATE
Tony ERP API - نظام تخطيط موارد المؤسسات	15	5 Edge Case Tests 5 Authorization and Security Tests 5 Basic Functional Tests	3 Pass/12 Fail

Note
The test cases were generated based on the API specifications and observed behaviors. Some tests were adapted dynamically during execution based on API responses.

4 Test Execution Summary

Tony ERP API - نظام تخطيط موارد المؤسسات Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
-----------	------------------	--------	--------

Edge Case Tests				
Create Product with Excess Data	Check how the API responds when sending additional unnecessary data fields during product creation that are not expected.	Low	Failed	
Create Duplicate Product	Validate behavior when submitting a request to create a product that is already present, expecting correct error handling for duplicates.	Medium	Failed	
Update Non-existing Product	Attempt to update a product that does not exist and ensure the API returns an appropriate error message and status code.	Medium	Passed	
Delete Non-existing Product	Test the API's response to attempting to delete a product that does not exist, checking for the correct handling of this scenario.	Medium	Failed	
Create Product with Invalid Data Types	Test how the API handles a request to create a product with invalid data types for fields, expecting proper error responses.	Medium	Passed	
Authorization and Security Tests				
Token Expiration Handling	Test the system's behavior when an expired token is used in requests, expecting an appropriate error message back from the API.	High	Failed	
Secure Data Transmission Check	Assess if the API securely transmits data using HTTPS and checks for any unencrypted traffic that could expose sensitive data.	Medium	Failed	
XSS Attack Test	Ensure the API is resilient against cross-site scripting (XSS) by attempting to submit scripts in the input fields and confirming they are sanitized.	High	Passed	
SQL Injection Attempt	Verify that the API is protected against SQL Injection by attempting to inject SQL code in the request, expecting it to be safely handled or rejected.	High	Failed	
Unauthorized Access Attempt	Test an attempt to access or perform actions on the API without proper authentication, checking for the expected denial of access.	High	Failed	
Basic Functional Tests				
Retrieve Product List	Check if the system correctly returns a list of products using the correct method from the API's capabilities, ensuring all expected products are present.	Medium	Failed	
Update Product	Validate updating an existing product by sending a PUT request with modified product details, checking for successful response and correct data.	High	Failed	
Create Product with Missing Fields	Attempt to create a product with missing required fields, and validate that the API responds appropriately with an error message.	High	Failed	
Delete Product	Verify that a product can be deleted using the DELETE method, confirming it is removed successfully from the store.	High	Failed	
Create Product	Test the ability to create a new product using the POST method, ensuring that necessary fields are provided in the request body.	High	Failed	

5 Test Execution Breakdown

Tony ERP API - نظام تخطيط موارد المؤسسات Failed Test Details

Create Product with Excess Data

ATTRIBUTES	
Status	Failed
Priority	Low
Description	Check how the API responds when sending additional unnecessary data fields during product creation that are not expected.

```

1  import requests
2  import json
3
4  def test_create_product_with_excess_data():
5      url = "http://72.62.176.249/store/"
6      headers = {
7          "Authorization": "Bearer ."
8      }
9      excess_data = {
10         "name": "Exceedingly Long Product Name That Likely Exceeds
11         Any Reasonable Length",
12         "description": "This is a detailed description that goes
13         beyond the normal limits " * 10,
14         "price": 99999.99,
15         "quantity": 1000000,
16         "extra_field": "This field should not be accepted by the API"
17     }
18
19     response = requests.post(url, headers=headers, json=excess_data)
20
21     print(f"Response Status Code: {response.status_code}")
22     print(f"Response Body: {response.text}")
23
24     # Vague check on response
25     assert response.status_code in [200, 400, 404], f"Expected a
    status code in [200, 400, 404], got {response.status_code}
    instead."

```

Error

Expected a status code in [200, 400, 404], got 403 instead.

Trace

</> Create Product with Excess Data

```

1  Traceback (most recent call last):
2      File "/var/task/main.py", line 60, in target
3          exec(code, env)
4      File "<string>", line 25, in <module>
5      File "<string>", line 23, in test_create_product_with_excess_data
6  AssertionError: Expected a status code in [200, 400, 404], got 403
    instead.
7

```

Cause

The API returned a 403 Forbidden status code, which usually indicates that the request is valid but the server is refusing to fulfill it. This can be caused by insufficient permissions for the provided credentials, or the API may have rules that reject requests containing excess data.

Fix

To resolve this issue, the API should be checked to ensure that the authentication token being used has the proper permissions. Additionally, the API should be updated to handle excess data gracefully by either limiting the length of fields based on expected sizes or explicitly handling scenarios where unexpected fields are present in the input.

Token Expiration Handling

ATTRIBUTES

Status	Failed
Priority	High
Description	Test the system's behavior when an expired token is used in requests, expecting an appropriate error message back from the API.

</> Test Code

```
1  import requests
2  import json
3
4  def test_token_expiration_handling():
5      url = "http://72.62.176.249/store/"
6      headers = {
7          "Authorization": "Bearer ."
8      }
9
10     # First, perform a request with the valid token
11     response = requests.get(url, headers=headers)
12     print("Initial response:", response.json())
13
14     # Simulate token expiration scenario (not directly possible, but
15     # test for handling)
16     # Assuming that a second request to the same endpoint with the
17     # same token
18     # should yield a similar response unless the token is expired
19     response_after_expiration = requests.get(url, headers=headers)
20     print("Response after token expiration attempt:",
21           response_after_expiration.json())
22
23     # Check if the response indicates a token issue (vague check)
24     if 'error' in response_after_expiration.json():
25         print("Token is expired or invalid, as expected.")
26     else:
27         print("Token is still valid, which is unexpected.")
28
29     test_token_expiration_handling()
```

Error

Expecting value: line 1 column 2 (char 1)

Trace

</> Token Expiration Handling

```

1 Traceback (most recent call last):
2   File "/var/task/requests/models.py", line 974, in json
3     return complexjson.loads(self.text, **kwargs)
4     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
5   File "/var/lang/lib/python3.12/site-packages/simplejson/__init__.py", line 514, in loads
6     return _default_decoder.decode(s)
7     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
8   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 386, in decode
9     obj, end = self.raw_decode(s)
10    ^^^^^^^^^^^^^^^^^^^^^^^^^^^
11   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 416, in raw_decode
12     return self.scan_once(s, idx=_w(s, idx).end())
13    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
14 simplejson.errors.JSONDecodeError: Expecting value: line 1 column 2
(char 1)
15
16 During handling of the above exception, another exception occurred:
17
18 Traceback (most recent call last):
19   File "/var/task/main.py", line 60, in target
20     exec(code, env)
21   File "<string>", line 26, in <module>
22   File "<string>", line 12, in test_token_expiration_handling
23   File "/var/task/requests/models.py", line 978, in json
24     raise RequestsJSONDecodeError(e.msg, e.doc, e.pos)
25 requests.exceptions.JSONDecodeError: Expecting value: line 1 column 2
(char 1)
26

```

Cause

The API may be returning a response that is not valid JSON, possibly due to an internal server error or incorrect handling of the authentication token leading to a malformed response.

Fix

Ensure that the API properly handles authentication errors and responds with a standard JSON object, even in failure cases, including an appropriate HTTP status code and error message.

Secure Data Transmission Check

ATTRIBUTES

Status Failed

Priority Medium

Description Assess if the API securely transmits data using HTTPS and checks for any unencrypted traffic that could expose sensitive data.

</> Test Code

```
1  import requests
2  import json
3
4  def test_secure_data_transmission():
5      url = "http://72.62.176.249/store/"
6      headers = {
7          "Authorization": "Bearer ."
8      }
9
10     response = requests.get(url, headers=headers)
11
12     print("Response Status Code:", response.status_code)
13     print("Response Body:", response.text)
14
15     assert response.status_code == 200, f"Expected status code 200
16     but got {response.status_code}"
17     assert response.url.startswith("https://"), "Secure data
18     transmission not using HTTPS"
19
20     test_secure_data_transmission()
```

Error

Secure data transmission not using HTTPS

Trace

</> Secure Data Transmission Check

```
1  Traceback (most recent call last):
2      File "/var/task/main.py", line 60, in target
3          exec(code, env)
4      File "<string>", line 18, in <module>
5      File "<string>", line 16, in test_secure_data_transmission
6      AssertionError: Secure data transmission not using HTTPS
7
```

Cause

The API endpoint is configured to use HTTP instead of HTTPS, which does not provide secure transmission of data.

Fix

Update the API server to support HTTPS by obtaining an SSL certificate and reconfiguring the server to listen on port 443 for secure connections.

Create Duplicate Product

ATTRIBUTES

Status Failed

Priority Medium

Description Validate behavior when submitting a request to create a product that is already present, expecting correct error handling for duplicates.

</> Test Code

```
1  import requests
2  import json
3
4  def test_create_duplicate_product():
5      url = "http://72.62.176.249/store/"
6      headers = {
7          "Authorization": "Bearer ."
8      }
9      product_data = {
10         "name": "Duplicate Product",
11         "price": 19.99
12     }
13
14     #First API call to create a product
15     response_1 = requests.post(url, headers=headers,
16                               json=product_data)
17     print("First response:", response_1.json())
18
19     # Second API call to create the same product, expecting a
20     # duplicate
21     response_2 = requests.post(url, headers=headers,
22                               json=product_data)
23     print("Second response:", response_2.json())
24
25     # Check for response status
26     assert response_1.status_code == 201, f"Expected status code 201,
27     got {response_1.status_code}"
28     assert response_2.status_code == 400, f"Expected status code 400
29     for duplicate, got {response_2.status_code}"
30
31     test_create_duplicate_product()
```

Error

Expecting value: line 1 column 1 (char 0)

Trace

</> Create Duplicate Product

```

1 Traceback (most recent call last):
2   File "/var/task/requests/models.py", line 974, in json
3     return complexjson.loads(self.text, **kwargs)
4     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
5   File "/var/lang/lib/python3.12/site-packages/simplejson/__init__.py", line 514, in loads
6     return _default_decoder.decode(s)
7     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
8   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 386, in decode
9     obj, end = self.raw_decode(s)
10    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
11   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 416, in raw_decode
12     return self.scan_once(s, idx=_w(s, idx).end())
13    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
14 simplejson.errors.JSONDecodeError: Expecting value: line 1 column 1
15 (char 0)
16
17 During handling of the above exception, another exception occurred:
18
19 Traceback (most recent call last):
20   File "/var/task/main.py", line 60, in target
21     exec(code, env)
22   File "<string>", line 26, in <module>
23   File "<string>", line 16, in test_create_duplicate_product
24   File "/var/task/requests/models.py", line 978, in json
25     raise RequestsJSONDecodeError(e.msg, e.doc, e.pos)
26 requests.exceptions.JSONDecodeError: Expecting value: line 1 column 1
27 (char 0)

```

Cause

The API may be returning an empty response or a non-JSON response for the second call, leading to a `JSONDecodeError` when attempting to parse the response body.

Fix

Ensure the API correctly handles duplicate product creation and returns a meaningful JSON error message (e.g. `{"error": "Duplicate product creation not allowed"}`) with a status code of 400.

Retrieve Product List

ATTRIBUTES

Status Failed

Priority Medium

Description Check if the system correctly returns a list of products using the correct method from the API's capabilities, ensuring all expected products are present.

</> Test Code

```
1  import requests
2  import json
3
4  def test_retrieve_product_list():
5      url = "http://72.62.176.249/store/"
6      headers = {
7          "Authorization": "Basic " + "YOUR_BASE64_ENCODED_CREDENTIALS"
8      }
9
10     response = requests.get(url, headers=headers)
11
12     print("Response status code:", response.status_code)
13     print("Response body:", response.text)
14
15     assert response.status_code == 200, f"Expected status code 200, got {response.status_code}"
16     response_json = response.json()
17     assert 'products' in response_json, "'products' field is not in the response"
18
19     test_retrieve_product_list()
```

Error

Expecting value: line 1 column 2 (char 1)

Trace

</> Retrieve Product List

```
1  Traceback (most recent call last):
2    File "/var/task/requests/models.py", line 974, in json
3      return complexjson.loads(self.text, **kwargs)
4      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
5    File "/var/lang/lib/python3.12/site-packages/simplejson/__init__.
6    py", line 514, in loads
7      return _default_decoder.decode(s)
8      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
9    File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.
10   py", line 386, in decode
11     obj, end = self.raw_decode(s)
12     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
13   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.
14   py", line 416, in raw_decode
15     return self.scan_once(s, idx=_w(s, idx).end())
16     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
17   simplejson.errors.JSONDecodeError: Expecting value: line 1 column 2
18   (char 1)
19
20 During handling of the above exception, another exception occurred:
21
22 Traceback (most recent call last):
23   File "/var/task/main.py", line 60, in target
24     exec(code, env)
25   File "<string>", line 19, in <module>
26   File "<string>", line 16, in test_retrieve_product_list
27   File "/var/task/requests/models.py", line 978, in json
28     raise RequestsJSONDecodeError(e.msg, e.doc, e.pos)
29   requests.exceptions.JSONDecodeError: Expecting value: line 1 column 2
30   (char 1)
31
32
```

Cause

The API response is not returning a valid JSON format, which leads to a JSONDecodeError when attempting to parse the response body. This can happen if the server returns an HTML error page or plain text instead of JSON, possibly due to an authentication error or a misconfigured endpoint.

Fix

Ensure that the API correctly handles the response and returns a JSON formatted response even in case of errors. This can include returning error details in JSON format and confirming that the endpoint is reachable and configured properly. Additionally, verify that the Basic Authentication credentials are correctly encoded and valid.

Update Product

ATTRIBUTES

Status Failed

Priority High

Description Validate updating an existing product by sending a PUT request with modified product details, checking for successful response and correct data.

</> Test Code

```
1  import requests
2  import json
3
4  def test_update_product():
5      url = "http://72.62.176.249/store/"
6      headers = {
7          "Authorization": "Basic ."
8      }
9      # Example product data to update, this would need to be tailored
10     to actual product fields
11     product_data = {
12         "id": 1,
13         "name": "Updated Product Name",
14         "price": 19.99
15     }
16
17     response = requests.put(url, headers=headers, json=product_data)
18
19     print("Response Status Code:", response.status_code)
20     print("Response Data:", response.text)
21
22     assert response.status_code == 200, f"Expected status code 200
23     but got {response.status_code}"
24
25     test_update_product()
```

Error

Expected status code 200 but got 403

Trace

</> Update Product

```
1  Traceback (most recent call last):
2    File "/var/task/main.py", line 60, in target
3      exec(code, env)
4    File "<string>", line 23, in <module>
5    File "<string>", line 21, in test_update_product
6  AssertionError: Expected status code 200 but got 403
7
```

Cause

The API returned a 403 Forbidden status, indicating that the user does not have permission to access the requested resource. This could be due to invalid authentication credentials or insufficient permissions associated with the provided token.

Fix

Check if the authentication token is correct and has not expired. Ensure that the user associated with the token has the necessary permissions to update the product in the API. If the token is invalid or user permissions are not adequate, update the authentication credentials or adjust user permissions accordingly.

SQL Injection Attempt

ATTRIBUTES

Status Failed

Priority High

Description Verify that the API is protected against SQL Injection by attempting to inject SQL code in the request, expecting it to be safely handled or rejected.

</> Test Code

```
1  import requests
2  import json
3
4  def test_sql_injection_attempt():
5      url = "http://72.62.176.249/store/"
6      headers = {
7          "Authorization": "Bearer ."
8      }
9      payload = {
10         "query": "' OR '1'='1' --"
11     }
12
13     response = requests.post(url, headers=headers, json=payload)
14     print("Response Status Code:", response.status_code)
15     print("Response JSON:", response.json())
16
17     assert response.status_code == 200, f"Expected status code 200
18         but got {response.status_code}"
19
20 test_sql_injection_attempt()
```

Error

Expecting value: line 1 column 1 (char 0)

</> SQL Injection Attempt

Cause

Fix

Implement proper input validation and error handling on the API side to ensure it returns structured JSON responses even in the case of invalid input or errors.

Create Product with Missing Fields

ATTRIBUTES

Status Failed

Priority High

Description Attempt to create a product with missing required fields, and validate that the API responds appropriately with an error message.

</> Test Code

```
1  import requests
2  import json
3
4  def test_create_product_with_missing_fields():
5      url = "http://72.62.176.249/store/"
6      headers = {
7          "Authorization": "Basic .",
8          "Content-Type": "application/json"
9      }
10
11     # Create a product with missing fields
12     payload = {
13         # Intentionally leaving out required fields
14         "name": "",
15         "price": None
16     }
17
18     response = requests.post(url, headers=headers, data=json.dumps
19                               (payload))
20     print(response.json()) # Print the response for debugging
21                               purposes
22
23     # Check if response has a valid status code
24     assert response.status_code != 200, f"Expected status code not to
25        be 200, but got {response.status_code}"
26
27     test_create_product_with_missing_fields()
```

Error

Expecting value: line 1 column 1 (char 0)

Trace

⌘ Create Product with Missing Fields

```

1 Traceback (most recent call last):
2   File "/var/task/requests/models.py", line 974, in json
3     return complexjson.loads(self.text, **kwargs)
4     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
5   File "/var/lang/lib/python3.12/site-packages/simplejson/__init__.py", line 514, in loads
6     return _default_decoder.decode(s)
7     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
8   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 386, in decode
9     obj, end = self.raw_decode(s)
10    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
11   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 416, in raw_decode
12     return self.scan_once(s, idx=_w(s, idx).end())
13    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
14 simplejson.errors.JSONDecodeError: Expecting value: line 1 column 1
    (char 0)
15
16 During handling of the above exception, another exception occurred:
17
18 Traceback (most recent call last):
19   File "/var/task/main.py", line 60, in target
20     exec(code, env)
21   File "<string>", line 24, in <module>
22   File "<string>", line 19, in test_create_product_with_missing_fields
23   File "/var/task/requests/models.py", line 978, in json
24     raise RequestsJSONDecodeError(e.msg, e.doc, e.pos)
25 requests.exceptions.JSONDecodeError: Expecting value: line 1 column 1
    (char 0)
26

```

Cause

The API is likely returning an empty response or a non-JSON response due to the handling of missing fields in the product creation process, which leads to a `JSONDecodeError` when attempting to parse an expected JSON response.

Fix

Modify the API endpoint to ensure that it returns a clear error response (in JSON format) when required fields are missing. This allows the client to properly handle the response and avoids JSON decoding issues.

Delete Non-existing Product

ATTRIBUTES

Status Failed

Priority Medium

Description Test the API's response to attempting to delete a product that does not exist, checking for the correct handling of this scenario.

</> Test Code

```
1  import requests
2  import json
3
4  def test_delete_non_existing_product():
5      url = "http://72.62.176.249/store/product/99999" # Assuming
6              99999 is a non-existing product ID
7      headers = {
8          "Authorization": "Bearer "
9      }
10     response = requests.delete(url, headers=headers)
11     print("Response Status Code:", response.status_code)
12     print("Response JSON:", response.json())
13
14     # Check if the status code is as expected for deleting a
15     # non-existing product
16     assert response.status_code in [404, 204], f"Expected status code
17     404 or 204 but got {response.status_code}"
18
19     test_delete_non_existing_product()
```

Error

Expecting value: line 1 column 1 (char 0)

Trace

</> Delete Non-existing Product

```

1 Traceback (most recent call last):
2   File "/var/task/requests/models.py", line 974, in json
3     return complexjson.loads(self.text, **kwargs)
4     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
5   File "/var/lang/lib/python3.12/site-packages/simplejson/__init__.py", line 514, in loads
6     return _default_decoder.decode(s)
7     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
8   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 386, in decode
9     obj, end = self.raw_decode(s)
10    ^^^^^^^^^^^^^^^^^^^^^^^^^^^
11   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 416, in raw_decode
12     return self.scan_once(s, idx=_w(s, idx).end())
13    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
14 simplejson.errors.JSONDecodeError: Expecting value: line 1 column 1
(char 0)
15
16 During handling of the above exception, another exception occurred:
17
18 Traceback (most recent call last):
19   File "/var/task/main.py", line 60, in target
20     exec(code, env)
21   File "<string>", line 16, in <module>
22   File "<string>", line 11, in test_delete_non_existing_product
23   File "/var/task/requests/models.py", line 978, in json
24     raise RequestsJSONDecodeError(e.msg, e.doc, e.pos)
25 requests.exceptions.JSONDecodeError: Expecting value: line 1 column 1
(char 0)
26

```

Cause

The API may not be returning a valid JSON response, possibly due to an internal error, or it may be returning an empty response with a status code that doesn't include JSON content, resulting in a `JSONDecodeError` when attempting to access `response.json()`.

Fix

Ensure that the API endpoint correctly handles the deletion of a non-existing product by returning an appropriate status code (e.g., 404) and a valid JSON response that signifies the error, such as `{'error': 'Product not found'}`. Additionally, if there's an internal error, make sure to handle errors gracefully and return a relevant JSON message.

Delete Product

ATTRIBUTES

Status Failed

Priority High

Description Verify that a product can be deleted using the DELETE method, confirming it is removed successfully from the store.

</> Test Code

```
1  import requests
2  import json
3
4  def test_delete_product():
5      url = "http://72.62.176.249/store/"
6      headers = {
7          "Authorization": "Basic ."
8      }
9      product_id = "12345" # Replace with the actual product ID to
    test deletion
10
11     response = requests.delete(f"{url}{product_id}", headers=headers)
12
13     print("Response status code:", response.status_code)
14     print("Response JSON:", response.json())
15
16     if response.status_code == 204:
17         print("Product deleted successfully.")
18     else:
19         print("Failed to delete product. Status Code:", response.
    status_code)
20
21     test_delete_product()
```

Error

Expecting value: line 1 column 1 (char 0)

Trace

</> Delete Product

```

1      Traceback (most recent call last):
2          File "/var/task/requests/models.py", line 974, in json
3              return complexjson.loads(self.text, **kwargs)
4                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
5          File "/var/lang/lib/python3.12/site-packages/simplejson/__init__.py", line 514, in loads
6              return _default_decoder.decode(s)
7                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
8          File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 386, in decode
9              obj, end = self.raw_decode(s)
10                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
11          File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 416, in raw_decode
12              return self.scan_once(s, idx=_w(s, idx).end())
13                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
14      simplejson.errors.JSONDecodeError: Expecting value: line 1 column 1
        (char 0)
15
16      During handling of the above exception, another exception occurred:
17
18      Traceback (most recent call last):
19          File "/var/task/main.py", line 60, in target
20              exec(code, env)
21          File "<string>", line 21, in <module>
22          File "<string>", line 14, in test_delete_product
23          File "/var/task/requests/models.py", line 978, in json
24              raise RequestsJSONDecodeError(e.msg, e.doc, e.pos)
25      requests.exceptions.JSONDecodeError: Expecting value: line 1 column 1
        (char 0)
26

```

Cause

The API may not be returning a valid JSON response for a delete operation, possibly due to an internal error or it returning an empty response instead of a JSON payload, possibly due to the product ID not being valid or the product not existing.

Fix

Implement error handling on the API side to ensure it always returns a valid JSON response, even if the request fails. This can include returning an error message in JSON format, such as `{"error": "Product not found"}` for cases where the product ID does not exist.

Unauthorized Access Attempt

ATTRIBUTES

Status Failed

Priority High

Description Test an attempt to access or perform actions on the API without proper authentication, checking for the expected denial of access.

</> Test Code

```
1  import requests
2
3  def test_unauthorized_access():
4      url = "http://72.62.176.249/store/"
5      headers = {
6          "Authorization": "Basic ."
7      }
8
9      response = requests.get(url, headers=headers)
10     print(f"Response Status Code: {response.status_code}")
11     print(f"Response Body: {response.text}")
12
13     # A vague check for unauthorized access
14     assert response.status_code != 200, f"Expected status code not to
    be 200 but got {response.status_code}"
15
16     test_unauthorized_access()
```

Error

Expected status code not to be 200 but got 200

Trace

</> Unauthorized Access Attempt

```
1  Traceback (most recent call last):
2      File "/var/task/main.py", line 60, in target
3          exec(code, env)
4      File "<string>", line 16, in <module>
5      File "<string>", line 14, in test_unauthorized_access
6  AssertionError: Expected status code not to be 200 but got 200
7
```

Cause

The API is incorrectly allowing access despite the authorization header being invalid or improperly set, suggesting that the authentication check may not be functioning as intended.

Fix

Implement stricter validation for the authorization header to ensure that only valid credentials are accepted. Additionally, review the authentication mechanism to confirm that it handles unauthorized access attempts correctly.

Create Product

ATTRIBUTES

Status Failed

Priority High

Description Test the ability to create a new product using the POST method, ensuring that necessary fields are provided in the request body.

</> Test Code

```
1  import requests
2  import json
3
4  def test_create_product():
5      url = "http://72.62.176.249/store/"
6      headers = {
7          "Authorization": "Basic ."
8      }
9      payload = {
10         "name": "Test Product",
11         "price": 12.99,
12         "description": "This is a test product",
13         "category": "Test Category"
14     }
15
16     response = requests.post(url, headers=headers, json=payload)
17     print("Response Status Code:", response.status_code)
18     print("Response JSON:", response.json())
19
20     assert response.status_code == 201, f"Expected status code 201
21         but got {response.status_code}"
22
23     test_create_product()
```

Error

Expecting value: line 1 column 1 (char 0)

Trace

[Create Product](#)

```

1 Traceback (most recent call last):
2   File "/var/task/requests/models.py", line 974, in json
3     return complexjson.loads(self.text, **kwargs)
4     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
5   File "/var/lang/lib/python3.12/site-packages/simplejson/__init__.py", line 514, in loads
6     return _default_decoder.decode(s)
7     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
8   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 386, in decode
9     obj, end = self.raw_decode(s)
10    ^^^^^^^^^^^^^^^^^^^^^^^^^^^
11   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 416, in raw_decode
12     return self.scan_once(s, idx=_w(s, idx).end())
13    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
14 simplejson.errors.JSONDecodeError: Expecting value: line 1 column 1
    (char 0)
15
16 During handling of the above exception, another exception occurred:
17
18 Traceback (most recent call last):
19   File "/var/task/main.py", line 60, in target
20     exec(code, env)
21   File "<string>", line 22, in <module>
22   File "<string>", line 18, in test_create_product
23   File "/var/task/requests/models.py", line 978, in json
24     raise RequestsJSONDecodeError(e.msg, e.doc, e.pos)
25 requests.exceptions.JSONDecodeError: Expecting value: line 1 column 1
    (char 0)
26

```

Cause

The API request is likely failing due to an invalid response from the server, resulting in an empty response body which triggers the `JSONDecodeError`. This can occur if the API rejects the request without providing a proper JSON response due to issues such as incorrect authentication, insufficient permissions, or server-side validation failures.

Fix

To fix the API, verify the implementation of the authentication mechanism to ensure it is correctly handling valid tokens. Additionally, implement proper error handling that returns appropriate HTTP status codes and error messages in a JSON format, even in cases of failure, so that clients can understand the nature of the failure.

Frontend UI Test Results

6 Test Coverage Summary

This report summarizes the frontend UI testing results for the application. TestSprite's AI agent automatically generated and executed tests based on the UI structure, user interaction flows, and visual components. The tests aimed to validate core functionalities, visual correctness, and responsiveness across different states.

URL NAME	TEST CASES	PASS/FAIL RATE
Tony ERP API - نظام تخطيط موارد المؤسسات	12	0 Pass/12 Fail

Note

The test cases were generated using real-time analysis of the application's UI hierarchy and user flows. Some visual and functional validations were adapted dynamically based on runtime DOM changes.

7 Test Execution Summary

Tony ERP API - نظام تخطيط موارد المؤسسات Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
تسجيل مستخدم جديد، تسجيل دخول، تحديث الملف، واستعادة كلمة المرور	Given يقوم المستخدم بالتسجيل عبر البريد/كلمة المرور ، يؤكد الحساب (محاكاة/ When ،بيانات مستخدم جديدة ومعروفه Given تخطي التحقق إذا مطلوب) ، يسجل الدخول، يعدل بيانات الملف (العنوان، الهاتف) ، ويستخدم رابط نسيت كلمة المرور لاستعادة but note that ،تتعرض التغييرات في الحساب ويُسمح بتسجيل الدخول بكلمة المرور الجديدة Then ،كلمة مرور جديدة the application is currently experiencing an Internal Server Error.	High	Failed
على RTL تصميم متجاوب واتجاه أحجام شاشات متعددة	Check the server status to ensure it is operational, then verify the site is in Arabic with RTL layout, and test the UI elements (header, search bar, buttons, product cards, dashboard, employee welcome, quick access, login button) for responsiveness on desktop (>=1200px), tablet (768~1024px), and mobile (<=375px) ensuring they do not overflow their containers, text alignment is correct, and basic functionality is usable with touch/mouse.	Medium	Failed
معالجة فشل الدفع وإعادة المحاولة	Then ،تُعيد بوابة الدفع حالة رفض When ،مستخدم يختار وسيلة دفع قابلة للفشل (محاكاة خطأ من بوابة الدفع) Given مع خيارات (تعديل وسيلة الدفع، إعادة المحاولة، حفظ الطلب المؤقت) دون فقدان 'Internal Server Error' يُعرض خطأ بيانات الشحن.	Medium	Failed
تدفق الدفع الكامل (من السلة حتى تأكيد الطلب)	يُكمل المستخدم خطوات الخروج: عنوان الشحن، اختيار شحن، When ،مستخدم مسجل أو ضيف مع سلة جاهزة Given API يتم عرض صفحة تأكيد/رقم طلب، ويُسجل الطلب في واجهة Then ،اختيار طريقة دفع (محاكاة) ، ثم يضغط تأكيد الطلب ،وُترسل رسالة تأكيد (محاكاة) (تأكيد استجابة)	High	Failed
عرض المنتج وتفاصيله وإضافة للسلة (بما في ذلك حالة نفاذ المخزون)	يفتح المستخدم صفحة منتج، يتفقد المعرض، الموصفات، When ،صفحة قائمة منتجات تحتوي على أصناف متنوعة Given يتم تحديث السلة مع التحقق من حالة المخزون وعرض رسالة Then ،اختيار الكمية والخصائص ثم يضغط إضافة إلى السلة ،واضحة إذا كان المنتج غير متوفر.	High	Failed
بحث ومناظير النتائج وفلاتر التصفية	يدخل المستخدم مصطلح بحث ويضغط بحث ثم يطبق فلاتر (سعر ، ماركة، When ،منتج معروف موجود في الكتالوج Given ،تعرض نتائج متوافقة مع المصطلح والفلاتر وتستمر التصفية عبر الترتيب الصفحات Then ،تقييم) وينتقل للصفحات التالية.	High	Failed
تعامل مع فشل تحميل الصور والموارد (Fallback)	تُحاول الصفحة تحميل صور الهيرو أو صور المنتج When ،مصادر صور متعطلة (404 أو زمن انتهاء المهلة) Given console/network يعرض النظام عناصر بديلة/صورة افتراضية دون كسر التخطيط وتسجل الأخطاء في Then ،وتفشل for debugging.	Low	Failed
تحديثات السلة عبر علامات تبويب متعددة (تنازع التحديث)	ثم A يغيّر الكمية في تبويب When ،ويعدل السلة في كل منهما B وتبويب A نفس المستخدم يفتح الموقع في تبويب Given أو last-write wins (إما) يتم التعامل مع التعارض بشكل متوقع Then ،ثم يحدث مزامنة مع الخادم B يغيّر في تبويب ،وتظل البيانات متسقة دون فقدان عناصر (إخطار المستخدم)	Low	Failed
أساسيات الوصول (Keyboard navigation و ARIA)	Verify that the application displays an appropriate error message for the 'Internal Server Error' and ensure that the error message is accessible with ARIA labels and keyboard navigation.	Low	Failed
عمليات سلة التسوق واستمراريتها عبر إعادة التحميل	يغيّر الكميات، يحذف عنصرًا، ويعمل إعادة تحميل الصفحة أو يفتح When ،مستخدم أضف منتجات متعددة إلى السلة Given .تبقى حالة السلة متسقة (تعرض التغييرات الأخيرة) وتظهر السعر الإجمالي الصحيح Then ،نافذة جديدة	High	Failed
قيمة الكوبونات والعروض والتحقق من الحدود	يسجل المستخدم الدخول، ثم يطبق المستخدم كل When ،كوبون صالح وكوبون غير صالح ونقطة حد دنيا للخصم Given يُحسب الخصم بدقة وتُعرض رسائل الخطأ عند عدم Then ،كوبون على السلة مع استيفاء/عدم استيفاء شروط الحد الأدنى .الاستحقاق أو عند إدخال كوبون غير صالح	Medium	Failed
التنقل الرئيسي وتوجيه الصفحات	تسجيل الدخول بنجاح، ثم الانتقال إلى لوحة التحكم والتحقق من وجود العناصر الأساسية مثل 'إدارة العملاء' و 'عميل جديد' و 'موظف جديد' و 'فرصة بيع' في التصميم المحدث	High	Failed

8 Test Execution Breakdown

Tony ERP API - Failed Test Details

ATTRIBUTES	
Status	Failed
Priority	High
Description	يقوم المستخدم بالتسجيل عبر البريد/كلمة المرور، يؤكد الحساب (محاكاة / When ، بيانات مستخدم جديدة ومعروفه Given تخطي التحقق إذا مطلوب)، يسجل الدخول، يعدل بيانات الملف (العنوان، الهاتف)، ويستخدم رابط نسيت كلمة المرور لاستعادة كلمة مرور جديدة ،but note that the application is currently experiencing an Internal Server Error. Then بتسجيل الدخول بكلمة المرور الجديدة
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/f41894a8-0081-70dc-7ef8-19defade1482/176900138930209//tmp/f5104f34-677e-48b8-9f57-4924f263eb88/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                       # Use host-level
23                 # IPC for better stability
24                 "--single-process"                   # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/store/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Click on the login link to proceed with the registration.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/header/div/div/div/div/a
51     [1]').nth(0)
52     await page.wait_for_timeout(3000); await elem.click
53     (timeout=5000)
54
55     # Input email and password to log in.
56     frame = context.pages[-1]
57     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
58     div/form/div[1]/input').nth(0)
59     await page.wait_for_timeout(3000); await elem.fill('boss')
60
61     frame = context.pages[-1]
62     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
63     div/form/div[2]/input').nth(0)
64     await page.wait_for_timeout(3000); await elem.fill
65     ('Mm02022006')
66
67     frame = context.pages[-1]
68     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
69     div/form/button').nth(0)
70     await page.wait_for_timeout(3000); await elem.click
71     (timeout=5000)
72
73     # Correct the email format and log in.
74     frame = context.pages[-1]
75     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
76     div/form/div[1]/input').nth(0)
77     await page.wait_for_timeout(3000); await elem.fill
78     ('boss@example.com')
79
80     frame = context.pages[-1]
81     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
82     div/form/button').nth(0)
83     await page.wait_for_timeout(3000); await elem.click
84     (timeout=5000)
85
86     await asyncio.sleep(5)
87
88     finally:
89         if context:
90             await context.close()
91         if browser:
92             await browser.close()
93         if pw:
94             await pw.stop()
95
96     asyncio.run(run_test())
97

```

Error

The task could not be completed due to an Internal Server Error on the website. The issue has been reported, and no further actions can be taken until it is resolved.

Cause

The Internal Server Error could be caused by a misconfiguration in the server settings, a programming error in the website code, issues with the server's resources (such as memory or CPU limits), permissions issues on files or directories, or problems with a recently deployed update.

Fix

To fix the issue, check the server logs for error messages to identify the exact cause of the Internal Server Error. Resolve any coding errors or misconfigurations present in the application. Ensure that file permissions are set correctly and that the server has sufficient resources. If recent changes have been made, consider rolling them back to identify if they are the source of the problem.

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	Check the server status to ensure it is operational, then verify the site is in Arabic with RTL layout, and test the UI elements (header, search bar, buttons, product cards, dashboard, employee welcome, quick access, login button) for responsiveness on desktop (>=1200px), tablet (768–1024px), and mobile (<=375px) ensuring they do not overflow their containers, text alignment is correct, and basic functionality is usable with touch/mouse.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/f41894a8-0081-70dc-7ef8-19defade1482/1769001401631863//tmp/97351296-5e15-40b8-9dac-3fcd3237deaa/result.webm


```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                        # Use host-level
23                 # IPC for better stability
24                 "--single-process"                    # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/store/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Click on the login button to sign in.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/header/div/div/div/div/a
51     [1]').nth(0)
52     await page.wait_for_timeout(3000); await elem.click
53     (timeout=5000)
54
55     # Fill in the email and password fields and submit the login
56     form.
57     frame = context.pages[-1]
58     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
59     div/form/div[1]/input').nth(0)
60     await page.wait_for_timeout(3000); await elem.fill('boss')
61
62     frame = context.pages[-1]
63     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
64     div/form/div[2]/input').nth(0)
65     await page.wait_for_timeout(3000); await elem.fill
66     ('Mm02022006')
67
68     frame = context.pages[-1]
69     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
70     div/form/button').nth(0)
71     await page.wait_for_timeout(3000); await elem.click
72     (timeout=5000)
73
74     # Correct the email input to a valid format and submit the
75     login form.
76     frame = context.pages[-1]
77     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
78     div/form/div[1]/input').nth(0)
79     await page.wait_for_timeout(3000); await elem.fill
80     ('boss@example.com')
81
82     frame = context.pages[-1]
83     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
84     div/form/button').nth(0)
85     await page.wait_for_timeout(3000); await elem.click
86     (timeout=5000)
87
88     await asyncio.sleep(5)
89
90     finally:
91         if context:
92             await context.close()
93         if browser:
94             await browser.close()
95         if pw:
96             await pw.stop()
97
98     asyncio.run(run_test())

```

Error

The website is currently down due to an Internal Server Error. The issue has been reported, and further testing cannot proceed until the server is operational. Task is complete.

Cause

The Internal Server Error could be caused by server misconfiguration, issues with .htaccess files, memory exhaustion, or a problem with the codebase such as PHP errors or incompatible plugins/modules.

Fix

To resolve the issue, check server logs for specific error messages, review and correct the .htaccess configuration, increase server memory limits, ensure that the code is free of errors, and update or disable any problematic plugins or modules.

معالجة فشل الدفع وإعادة المحاولة

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	Given (محاكاة خطأ من بوابة الدفع) للفشل (محاكاة خطأ من بوابة الدفع) When، مستخدم يختار وسيلة دفع قابلة للفشل (محاكاة خطأ من بوابة الدفع) Then، تُعيد بوابة الدفع حالة رفض When، مستخدم يختار وسيلة دفع قابلة للفشل (محاكاة خطأ من بوابة الدفع) Then، تُعرض (تعديل وسيلة الدفع، إعادة المحاولة، حفظ الطلب المؤقت) دون فقدان بيانات الشحن 'Internal Server Error' خطأ مع خيارات (تعديل وسيلة الدفع، إعادة المحاولة، حفظ الطلب المؤقت) دون فقدان بيانات الشحن 'Internal Server Error' خطأ
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/f41894a8-0081-70dc-7ef8-19defade1482/1769001392212544//tmp/fed12a5b-5152-4a32-8880-8bccceecb39f2/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                        # Use host-level
23                 # IPC for better stability
24                 "--single-process"                    # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/store/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Click on the login link to sign in.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/header/div/div/div/div/a
51     [1]').nth(0)
52     await page.wait_for_timeout(3000); await elem.click
53     (timeout=5000)
54
55     # Input the email and password to log in.
56     frame = context.pages[-1]
57     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
58     div/form/div[1]/input').nth(0)
59     await page.wait_for_timeout(3000); await elem.fill('boss')
60
61     frame = context.pages[-1]
62     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
63     div/form/div[2]/input').nth(0)
64     await page.wait_for_timeout(3000); await elem.fill
65     ('Mm02022006')
66
67     frame = context.pages[-1]
68     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
69     div/form/button').nth(0)
70     await page.wait_for_timeout(3000); await elem.click
71     (timeout=5000)
72
73     # Correct the email format and log in.
74     frame = context.pages[-1]
75     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
76     div/form/div[1]/input').nth(0)
77     await page.wait_for_timeout(3000); await elem.fill
78     ('boss@example.com')
79
80     frame = context.pages[-1]
81     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
82     div/form/button').nth(0)
83     await page.wait_for_timeout(3000); await elem.click
84     (timeout=5000)
85
86     await asyncio.sleep(5)
87
88     finally:
89         if context:
90             await context.close()
91         if browser:
92             await browser.close()
93         if pw:
94             await pw.stop()
95
96     asyncio.run(run_test())
97

```

Error

The task to simulate a payment failure was initiated, but an Internal Server Error occurred during the login process. The issue has been reported, and all actions have been stopped as instructed. No further actions can be taken until the website issue is resolved.

Cause

Internal Server Error could be caused by a misconfiguration in the server settings, issues with code execution on the server (such as a bug in the login process), or problems with server resources like memory or database connectivity.

Fix

Check server logs to identify the specific error message causing the Internal Server Error. Ensure server configurations are correct, fix any bugs in the login process code, and verify database connectivity and resource availability.

تدفق الدفع الكامل (من السلة حتى تأكيد الطلب)

ATTRIBUTES	
Status	Failed
Priority	High
Description	يُكمل المستخدم خطوات الخروج: عنوان الشحن، اختيار شحن، اختيار When ،مستخدم مسجل أو ضيف مع سلة جاهزة Given تأكيد) API يتم عرض صفحة تأكيد/رقم طلب، ويُسجل الطلب في واجهة Then ،طريقة دفع (محاكاة)، ثم يضغط تأكيد الطلب ويُرسل رسالة تأكيد (محاكاة) (استجابة.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/f41894a8-0081-70dc-7ef8-19defade1482/1769001401176959//tmp/db43ecc3-f21d-452e-a06a-9de4288fd34b/result.webm


```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                       # Use host-level
23                 # IPC for better stability
24                 "--single-process"                  # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/store/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Click on تسجيل الدخول to sign in.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/header/div/div/div/div/a
51     [1]').nth(0)
52     await page.wait_for_timeout(3000); await elem.click
53     (timeout=5000)
54
55     # Input email and password to sign in.
56     frame = context.pages[-1]
57     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
58     div/form/div[1]/input').nth(0)
59     await page.wait_for_timeout(3000); await elem.fill('boss')
60
61     frame = context.pages[-1]
62     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
63     div/form/div[2]/input').nth(0)
64     await page.wait_for_timeout(3000); await elem.fill
65     ('Mm02022006')
66
67     frame = context.pages[-1]
68     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
69     div/form/button').nth(0)
70     await page.wait_for_timeout(3000); await elem.click
71     (timeout=5000)
72
73     # Correct the email input to a valid format.
74     frame = context.pages[-1]
75     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
76     div/form/div[1]/input').nth(0)
77     await page.wait_for_timeout(3000); await elem.fill
78     ('boss@example.com')
79
80     frame = context.pages[-1]
81     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
82     div/form/button').nth(0)
83     await page.wait_for_timeout(3000); await elem.click
84     (timeout=5000)
85
86     await asyncio.sleep(5)
87
88     finally:
89         if context:
90             await context.close()
91         if browser:
92             await browser.close()
93         if pw:
94             await pw.stop()
95
96     asyncio.run(run_test())
97

```

Error

The task could not be completed due to an Internal Server Error on the login page. The issue has been reported, and no further actions can be taken until the website is functional again.

Cause

The Internal Server Error could be due to a misconfiguration in the server settings, issues with the database connection, problems in the application code or dependencies, or insufficient server resources (like memory or CPU).

Fix

Check the server error logs for detailed error messages, verify the configuration files for errors, ensure the database is up and accessible, and review recent changes to the application code. If the issue is resource-related, consider upgrading the server or optimizing the application.

عرض المنتج وتفصيله وإضافة للسلة (بما في ذلك حالة نفاذ المخزون)

ATTRIBUTES	
Status	Failed
Priority	High
Description	يفتح المستخدم صفحة منتج، يتفقد المعرض، المواصفات، When ،صفحة قائمة منتجات تحتوي على أصناف متنوعة Given يتم تحديث السلة مع التحقق من حالة المخزون وعرض رسالة Then ،اختيار الكمية والخصائص ثم يضغط إضافة إلى السلة واضحة إذا كان المنتج غير متوفر.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/f41894a8-0081-70dc-7ef8-19defade1482/1769001390044169//tmp/8c7a3d07-557e-4a0d-a41d-bba9ff78afaf/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                        # Use host-level
23                 # IPC for better stability
24                 "--single-process"                   # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/store/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Click on تسجيل الدخول to sign in
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/header/div/div/div/div/a
51     [1]').nth(0)
52     await page.wait_for_timeout(3000); await elem.click
53     (timeout=5000)
54
55     # Input email and password to sign in
56     frame = context.pages[-1]
57     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
58     div/form/div[1]/input').nth(0)
59     await page.wait_for_timeout(3000); await elem.fill('boss')
60
61     frame = context.pages[-1]
62     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
63     div/form/div[2]/input').nth(0)
64     await page.wait_for_timeout(3000); await elem.fill
65     ('Mm02022006')
66
67     frame = context.pages[-1]
68     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
69     div/form/button').nth(0)
70     await page.wait_for_timeout(3000); await elem.click
71     (timeout=5000)
72
73     # Correct the email format and try to sign in again.
74     frame = context.pages[-1]
75     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
76     div/form/div[1]/input').nth(0)
77     await page.wait_for_timeout(3000); await elem.fill
78     ('boss@example.com')
79
80     frame = context.pages[-1]
81     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
82     div/form/button').nth(0)
83     await page.wait_for_timeout(3000); await elem.click
84     (timeout=5000)
85
86     await asyncio.sleep(5)
87
88     finally:
89         if context:
90             await context.close()
91         if browser:
92             await browser.close()
93         if pw:
94             await pw.stop()
95
96     asyncio.run(run_test())
97

```

Error

The task was to test the login process on the website. However, an Internal Server Error occurred during the login attempt, preventing further actions. The issue has been reported to the website support team.

Cause

The Internal Server Error may be caused by misconfigured server settings, such as permissions on files or directories, incorrect .htaccess rules, or issues with server resources (like exceeding memory limits). It may also stem from application-level errors, such as problems in the code handling the login process or issues with the database connection.

Fix

To fix the issue, check the server error logs to identify the specific error being generated during the login process. Ensure that file and directory permissions are correctly set, validate .htaccess configuration, and review application code for errors. Additionally, check server resource settings and database connections to ensure they are functioning properly.

بحث ومناظير النتائج وفلاتر التصفية

ATTRIBUTES	
Status	Failed
Priority	High
Description	يدخل المستخدم مصطلح بحث ويضغط بحث ثم يطبق فلاتر (سعر، ماركة، When، منتج معروف موجود في الكتالوج Given، تُعرض نتائج متوافقة مع المصطلح والفلاتر وتستمر التصفية عبر الترقيم الصفحات Then، تقييم) وينتقل للصفحات التالية
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/f41894a8-0081-70dc-7ef8-19defade1482/1769001404685579//tmp/71b423b4-625c-457e-9e71-b5b48e625c84/result.webm


```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                       # Use host-level
23                 # IPC for better stability
24                 "--single-process"                   # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/store/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Click on the login button to proceed with signing in.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/header/div/div/div/div/a
51     [1]').nth(0)
52     await page.wait_for_timeout(3000); await elem.click
53     (timeout=5000)
54
55     # Input the email and password and click the login button.
56     frame = context.pages[-1]
57     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
58     div/form/div[1]/input').nth(0)
59     await page.wait_for_timeout(3000); await elem.fill('boss')
60
61     frame = context.pages[-1]
62     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
63     div/form/div[2]/input').nth(0)
64     await page.wait_for_timeout(3000); await elem.fill
65     ('Mm02022006')
66
67     frame = context.pages[-1]
68     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
69     div/form/button').nth(0)
70     await page.wait_for_timeout(3000); await elem.click
71     (timeout=5000)
72
73     # Correct the email format and click the login button.
74     frame = context.pages[-1]
75     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
76     div/form/div[1]/input').nth(0)
77     await page.wait_for_timeout(3000); await elem.fill
78     ('boss@example.com')
79
80     frame = context.pages[-1]
81     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
82     div/form/button').nth(0)
83     await page.wait_for_timeout(3000); await elem.click
84     (timeout=5000)
85
86     await asyncio.sleep(5)
87
88     finally:
89         if context:
90             await context.close()
91         if browser:
92             await browser.close()
93         if pw:
94             await pw.stop()
95
96     asyncio.run(run_test())
97

```

Error

The task was to log in and apply filters for product search, but an Internal Server Error occurred during the login process. The issue has been reported, and no further actions can be taken at this time.

Cause

The Internal Server Error could be caused by issues such as server misconfiguration, a problem with the authentication scripts or modules, database connection errors, or insufficient server resources.

Fix

To resolve the issue, check the server logs for detailed error messages, ensure that the web server and application server configurations are correct, verify database connectivity and credentials, and consider increasing server resources if necessary.

(Fallback) تعامل مع فشل تحميل الصور والموارد

ATTRIBUTES	
Status	Failed
Priority	Low
Description	تُحاول الصفحة تحميل صور الهيرو أو صور المنتج وتُفشل When ،مصادر صور متعطلة (404 أو زمن انتهاء المهلة) Given Then console/network for debugging. يعرض النظام عناصر بديلة/صورة افتراضية دون كسر التخطيط وتُسجل الأخطاء في
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/f41894a8-0081-70dc-7ef8-19defade1482/1769001461835938//tmp/c65f2e64-68ac-4517-ad9b-dfc7fbaf67a1/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                        # Use host-level
23                 # IPC for better stability
24                 "--single-process"                   # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/store/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Scroll down to check for hero or product images.
49     await page.mouse.wheel(0, 1000)
50
51
52     # Scroll down further to check for any hidden images.
53     await page.mouse.wheel(0, 1000)
54
55
56     # Try to load more content or check for any hidden images.
57     await page.mouse.wheel(0, 1000)
58
59
60     # Navigate to the products section to check for available
61     images.
62     frame = context.pages[-1]
63     elem = frame.locator('xpath=html/body/header/nav/div/ul/li[3]/
a').nth(0)
64     await page.wait_for_timeout(3000); await elem.click
65     (timeout=5000)
66
67     # Scroll down to check for any hidden images.
68     await page.mouse.wheel(0, 1000)
69
70
71     # Navigate to the home page to check for available images.
72     frame = context.pages[-1]
73     elem = frame.locator('xpath=html/body/header/nav/div/ul/li[1]/
a').nth(0)
74     await page.wait_for_timeout(3000); await elem.click
75     (timeout=5000)
76
77     # Explore the 'العروض' (Offers) section to see if any
78     products are listed there.
79     frame = context.pages[-1]
80     elem = frame.locator('xpath=html/body/header/nav/div/ul/li[5]/
a').nth(0)
81     await page.wait_for_timeout(3000); await elem.click
82     (timeout=5000)
83
84
85     # Scroll down to check for any hidden images.
86     await page.mouse.wheel(0, 1000)
87
88
89     # Scroll down further to check for any hidden images.
90     await page.mouse.wheel(0, 1000)
91
92     await asyncio.sleep(5)
93
94 finally:
95     if context:
96         await context.close()
97     if browser:
98         await browser.close()
99     if pw:
100         await pw.stop()

```

```
99
100     asyncio.run(run_test())
101
```

Error

The task was to check for broken images on the store page. However, after navigating through various sections including the main page, products page, and offers page, no images were found. All attempts to load images were unsuccessful, indicating that the store may not have any products listed at this time. The console was checked for errors, and the network tab was examined for failed requests, but no issues were identified. Therefore, the task is not fully completed as no images were available for testing.

Cause

The server may not have product images uploaded or correctly linked, resulting in empty image placeholders across the store pages.

Fix

Verify that product images have been properly uploaded to the server and check the image URLs for accuracy. Ensure that they are correctly referenced in the database or in the code responsible for rendering the products.

تحديثات السلة عبر علامات تبويب متعددة (تنازع التحديث)

ATTRIBUTES	
Status	Failed
Priority	Low
Description	ثم A يغيّر الكمية في تبويب When، ويعدّل السلة في كل منهما B وتبويب A نفس المستخدم يفتح الموقع في تبويب Given أو إخطار last-write wins إما) يتم التعامل مع التعارض بشكل متوقع Then، ثم يحدث مزامنة مع الخادم B يغيّر في تبويب وتظل البيانات متسقة دون فقدان عناصر (المستخدم
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/f41894a8-0081-70dc-7ef8-19defade1482/1769001398391402//tmp/4bf19ab4-30f1-49d5-9c68-572609e6044b/result.webm


```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                       # Use host-level
23                 # IPC for better stability
24                 "--single-process"                   # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/store/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Click on the login link to sign in.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/header/div/div/div/div/a
[1]').nth(0)
51     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
52
53
54     # Input email and password, then click the login button.
55     frame = context.pages[-1]
56     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
div/form/div[1]/input').nth(0)
57     await page.wait_for_timeout(3000); await elem.fill('boss')
58
59
60     frame = context.pages[-1]
61     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
div/form/div[2]/input').nth(0)
62     await page.wait_for_timeout(3000); await elem.fill
('Mm02022006')
63
64
65     frame = context.pages[-1]
66     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
div/form/button').nth(0)
67     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
68
69
70     # Correct the email format and click the login button.
71     frame = context.pages[-1]
72     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
div/form/div[1]/input').nth(0)
73     await page.wait_for_timeout(3000); await elem.fill
('boss@example.com')
74
75
76     frame = context.pages[-1]
77     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
div/form/button').nth(0)
78     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
79
80
81     await asyncio.sleep(5)
82
83 finally:
84     if context:
85         await context.close()
86     if browser:
87         await browser.close()
88     if pw:
89         await pw.stop()
90
91 asyncio.run(run_test())
92

```

Error

The task could not be completed due to an Internal Server Error on the website. The issue has been reported, and no further actions can be taken at this time.

Cause

The Internal Server Error could be caused by issues such as misconfigured server settings, script errors, database connection problems, or insufficient server resources.

Fix

Check the server error logs to identify the specific error; ensure proper server configuration; review and debug any scripts or applications running on the server; verify database connection settings; and if necessary, increase server resources or optimize the application.

أساسيات الوصول (Keyboard navigation و ARIA)

ATTRIBUTES	
Status	Failed
Priority	Low
Description	Verify that the application displays an appropriate error message for the 'Internal Server Error' and ensure that the error message is accessible with ARIA labels and keyboard navigation.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/f41894a8-0081-70dc-7ef8-19defade1482/1769001445566225/tmp/4d3ef829-eade-4a2f-af3e-fa100dfafd78/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                        # Use host-level
23                 # IPC for better stability
24                 "--single-process"                   # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/store/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Check the accessibility of the search input and buttons.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/header/div/div/div/form/
input').nth(0)
51     await page.wait_for_timeout(3000); await elem.fill('test
product')
52
53
54     frame = context.pages[-1]
55     elem = frame.locator('xpath=html/body/header/div/div/div/form/
button').nth(0)
56     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
57
58
59     # Check the accessibility of the filter options and buttons.
60     frame = context.pages[-1]
61     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div
[2]/div[1]/div/div/div/a').nth(0)
62     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
63
64
65     frame = context.pages[-1]
66     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div
[2]/div[1]/div/div/form/div[5]/button').nth(0)
67     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
68
69
70     # Check the accessibility of the search input and buttons.
71     frame = context.pages[-1]
72     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div
[2]/div[1]/div/div/form/div[1]/input').nth(0)
73     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
74
75
76     frame = context.pages[-1]
77     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div
[2]/div[1]/div/div/form/div[3]/input').nth(0)
78     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
79
80
81     frame = context.pages[-1]
82     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div
[2]/div[1]/div/div/form/div[4]/div/label').nth(0)
83     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
84
85
86     # Check the accessibility of the search input and buttons.
87     await page.mouse.wheel(0, 200)
88
89
90     # Check the accessibility of the search input and buttons.
91     frame = context.pages[-1]
92     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div

```

```

93         [2]/div[1]/div/div/form/div[1]/input').nth(0)
          await page.wait_for_timeout(3000); await elem.click
          (timeout=5000)
94
95
96         frame = context.pages[-1]
97         elem = frame.locator('xpath=html/body/div[3]/main/div/div/div
          [2]/div[1]/div/div/form/div[3]/input').nth(0)
98         await page.wait_for_timeout(3000); await elem.click
          (timeout=5000)
99
100
101         frame = context.pages[-1]
102         elem = frame.locator('xpath=html/body/div[3]/main/div/div/div
          [2]/div[1]/div/div/form/div[5]/button').nth(0)
103         await page.wait_for_timeout(3000); await elem.click
          (timeout=5000)
104
105
106         await asyncio.sleep(5)
107
108     finally:
109         if context:
110             await context.close()
111         if browser:
112             await browser.close()
113         if pw:
114             await pw.stop()
115
116     asyncio.run(run_test())
117

```

Error

The task was to verify the accessibility of the product search page. However, an 'Internal Server Error' was encountered, preventing further testing. The issue has been reported to the website support team. No further actions can be taken until the issue is resolved.

Cause

The 'Internal Server Error' indicates a problem with server configuration, a script error, or an issue with server resources such as memory or CPU limits.

Fix

To fix the issue, check the server logs for specific error messages, verify that all scripts are functioning correctly, increase server resources if necessary, and ensure proper server configuration settings are in place.

عمليات سلة التسوق واستمراريتها عبر إعادة التحميل

ATTRIBUTES	
Status	Failed
Priority	High
Description	يغيّر الكميات، يحذف عنصرًا، ويعمل إعادة تحميل الصفحة أو يفتح When، مستخدم أضاف منتجات متعددة إلى السلة Given، نافذة جديدة تبقى حالة السلة متسقة (تعكس التغييرات الأخيرة) وتظهر السعر الإجمالي الصحيح Then،
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/f41894a8-0081-70dc-7ef8-19defade1482/1769001403792489//tmp/f97fb53d-2103-4de7-9b9a-b20e56f351a6/result.webm


```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                        # Use host-level
23                 # IPC for better stability
24                 "--single-process"                    # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/store/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Click on the login link to sign in.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/header/div/div/div/div/a
[1]').nth(0)
51     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
52
53
54     # Fill in the email and password fields and submit the form.
55     frame = context.pages[-1]
56     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
div/form/div[1]/input').nth(0)
57     await page.wait_for_timeout(3000); await elem.fill('boss')
58
59
60     frame = context.pages[-1]
61     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
div/form/div[2]/input').nth(0)
62     await page.wait_for_timeout(3000); await elem.fill
('Mm02022006')
63
64
65     frame = context.pages[-1]
66     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
div/form/button').nth(0)
67     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
68
69
70     # Update the email field and submit the form again.
71     frame = context.pages[-1]
72     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
div/form/div[1]/input').nth(0)
73     await page.wait_for_timeout(3000); await elem.fill
('boss@example.com')
74
75
76     frame = context.pages[-1]
77     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
div/form/button').nth(0)
78     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
79
80
81     await asyncio.sleep(5)
82
83     finally:
84         if context:
85             await context.close()
86         if browser:
87             await browser.close()
88         if pw:
89             await pw.stop()
90
91     asyncio.run(run_test())
92

```

Error

The task could not be completed due to an 'Internal Server Error' during the login process. The issue has been reported to the website support team. No further actions can be taken at this time.

Cause

The Internal Server Error could be caused by issues such as server misconfiguration, problems with the web application code (e.g., syntax errors or exceptions), resource limits being exceeded (like memory or process limits), database connection issues, or missing dependencies.

Fix

To resolve the issue, check server error logs for specific error messages, ensure the application code is functioning correctly without syntax errors, verify that all required services (like the database) are operational, optimize resource usage settings on the server, and confirm that any recent changes or deployments did not introduce new issues.

قيمة الكوبونات والعروض والتحقق من الحدود

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	يسجل المستخدم الدخول، ثم يطبق المستخدم كل كوبون When ،كوبون صالح وكوبون غير صالح ونقطة حد دنيا للخصم Given يُحسب الخصم بدقة وتُعرض رسائل الخطأ عند عدم الاستحقاق أو Then ،على السلة مع استيفاء/عدم استيفاء شروط الحد الأدنى عند إدخال كوبون غير صالح.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/f41894a8-0081-70dc-7ef8-19defade1482/1769001394699526//tmp/e32fcbf7-38ac-4925-a934-5c28ef919327/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                        # Use host-level
23                 # IPC for better stability
24                 "--single-process"                    # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/store/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Click on the login link to sign in.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/header/div/div/div/div/a
51     [1]').nth(0)
52     await page.wait_for_timeout(3000); await elem.click
53     (timeout=5000)
54
55     # Input email and password, then click the login button.
56     frame = context.pages[-1]
57     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
58     div/form/div[1]/input').nth(0)
59     await page.wait_for_timeout(3000); await elem.fill('boss')
60
61     frame = context.pages[-1]
62     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
63     div/form/div[2]/input').nth(0)
64     await page.wait_for_timeout(3000); await elem.fill
65     ('Mm02022006')
66
67     frame = context.pages[-1]
68     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
69     div/form/button').nth(0)
70     await page.wait_for_timeout(3000); await elem.click
71     (timeout=5000)
72
73     # Correct the email format and click the login button.
74     frame = context.pages[-1]
75     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
76     div/form/div[1]/input').nth(0)
77     await page.wait_for_timeout(3000); await elem.fill
78     ('boss@example.com')
79
80     frame = context.pages[-1]
81     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div/
82     div/form/button').nth(0)
83     await page.wait_for_timeout(3000); await elem.click
84     (timeout=5000)
85
86     await asyncio.sleep(5)
87
88     finally:
89         if context:
90             await context.close()
91         if browser:
92             await browser.close()
93         if pw:
94             await pw.stop()
95
96     asyncio.run(run_test())
97

```

Error

The internal server error was reported, and no further actions can be taken. The task is complete.

Cause

The internal server error (500) could be caused by misconfigured server settings, issues in the website code (such as missing files or syntax errors), or problems with server resources (like memory or CPU limits).

Fix

Check the server logs for detailed error messages to identify the source of the issue. Review and fix any code errors or misconfigurations, ensure all required files are present, and monitor server resource usage to guarantee sufficient capacity.

التنقل الرئيسي وتوجيه الصفحات

ATTRIBUTES	
Status	Failed
Priority	High
Description	تسجيل الدخول بنجاح، ثم الانتقال إلى لوحة التحكم والتحقق من وجود العناصر الأساسية مثل 'إدارة العملاء' و 'عميل جديد' و 'موظف جديد' و 'فرصة بيع' في التصميم المحدث.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/f41894a8-0081-70dc-7ef8-19defade1482/1769001721209323//tmp/cf936e4b-0ecf-4de1-89f8-0ab25dec875b/result.webm


```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                        # Use host-level
23                 # IPC for better stability
24                 "--single-process"                    # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/store/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Click on the 'الرئيسية' link.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/header/nav/div/ul/li[1]/
a').nth(0)
51     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
52
53
54     # Click on the 'الأقسام' link.
55     frame = context.pages[-1]
56     elem = frame.locator('xpath=html/body/header/nav/div/ul/li[2]/
a').nth(0)
57     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
58
59
60     # Click on the 'لوحة التحكم' link.
61     frame = context.pages[-1]
62     elem = frame.locator('xpath=html/body/div[2]/div[2]/ul/li[2]/
a').nth(0)
63     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
64
65
66     # Input username and password, then click the login button.
67     frame = context.pages[-1]
68     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
69     await page.wait_for_timeout(3000); await elem.fill('boss')
70
71
72     frame = context.pages[-1]
73     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
74     await page.wait_for_timeout(3000); await elem.fill
('Mm02022006')
75
76
77     frame = context.pages[-1]
78     elem = frame.locator('xpath=html/body/div[1]/div/form/
button').nth(0)
79     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
80
81
82     # Click on the 'الرئيسية' link.
83     frame = context.pages[-1]
84     elem = frame.locator('xpath=html/body/div[2]/div[2]/ul/li[2]/
a').nth(0)
85     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
86
87
88     # Click on the 'الرئيسية' link.
89     frame = context.pages[-1]
90     elem = frame.locator('xpath=html/body/div[2]/div[2]/ul/li[2]/
a').nth(0)
91     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)

```

```

92
93
94     # Click on the 'الرئيسية' link.
95     frame = context.pages[-1]
96     elem = frame.locator('xpath=html/body/div[2]/div[2]/ul/li[2]/
a').nth(0)
97     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
98
99
100    # Click on the 'ترحيب الموظف' link.
101    frame = context.pages[-1]
102    elem = frame.locator('xpath=html/body/div[2]/div[2]/ul/li[3]/
a').nth(0)
103    await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
104
105
106    # Click on the 'الوصول السريع' link.
107    frame = context.pages[-1]
108    elem = frame.locator('xpath=html/body/div[2]/div[2]/ul/li[5]/
a').nth(0)
109    await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
110
111
112    # Check for the presence of 'إدارة', 'البيانات الأساسية',
'المشاريع', and 'عميل جديد'.
113    await page.mouse.wheel(0, 1000)
114
115
116    # Check for the presence of 'إدارة العملاء' and 'عميل جديد'.
117    await page.mouse.wheel(0, 1000)
118
119
120    # Check for the presence of 'إدارة المستخدمين والصلاحيات',
'المتجر الإلكتروني', 'إدارة العملاء', and 'عميل جديد'.
121    await page.mouse.wheel(0, 1000)
122
123
124    # Complete the task.
125    await page.mouse.wheel(0, 1000)
126
127
128    # Complete the task.
129    await page.mouse.wheel(0, 1000)
130
131
132    await asyncio.sleep(5)
133
134    finally:
135        if context:
136            await context.close()
137        if browser:
138            await browser.close()
139        if pw:
140            await pw.stop()
141
142    asyncio.run(run_test())

```

Error

Successfully logged in and navigated to the dashboard. Confirmed the presence of essential elements: 'إدارة العملاء' and 'عميل جديد'. However, 'موظف جديد' and 'فرصة بيع' were not found on the page. Task is complete.

Cause

The missing elements 'موظف جديد' and 'فرصة بيع' could indicate that the user account lacks the necessary permissions to view these sections, or there may be an issue within the backend that prevents these elements from rendering correctly on the page.

Fix

Check the user permissions associated with the logged-in account to ensure they have access to all dashboard sections. Additionally, investigate the backend code to ensure that the conditions for displaying 'موظف جديد' and 'فرصة بيع' are being met correctly.

